

Penetration Testing Notes

Week 3 — Nmap, OWASP ZAP, and manual attack walk-throughs

Author: Maryam Mueen
Intern ID: DHC-1151

Authorisation. All testing was performed against locally-hosted instances on `localhost:3000` owned by the intern. No third-party systems were touched.

This document covers the offensive testing performed against both the vulnerable build (`vulnerable-app/`) and the hardened build (`secure-app/`). The toolset is intentionally small and beginner-friendly: OWASP ZAP, Nmap, and `curl` with browser developer tools.

1 Reconnaissance with Nmap

1.1 Port and service scan

```
nmap -sV -p 3000 -T4 localhost
```

Expected output against the running app:

```
PORT      STATE SERVICE VERSION
3000/tcp  open  http    Node.js Express framework
```

1.2 HTTP-method probe

```
nmap -p 3000 --script http-methods --script-args http-methods.url-path=/login localhost
```

Confirms which verbs the endpoint accepts (GET, POST). We expect no TRACE, PUT, or DELETE — they are indeed absent.

1.3 Vulnerability-script sweep

```
nmap -p 3000 --script "http-enum,http-headers,http-csrf,http-xssed,http-sql-injection" localhost
```

Against `vulnerable-app/`, the `http-headers` script reports the missing `Strict-Transport-Security`, `X-Frame-Options`, and `Content-Security-Policy` headers, matching finding V-04. Against `secure-app/`, those scripts come back clean because `helmet()` adds the headers.

2 Active Web Scanning with OWASP ZAP

2.1 Set-up

1. Install ZAP from <https://www.zaproxy.org/download/>.
2. Start ZAP, choose **Quick Start** then **Automated Scan**.
3. Target URL: `http://localhost:3000`.
4. Use the traditional spider plus the AJAX spider, then run the Active Scan.

2.2 Findings against vulnerable-app/

ZAP alert	Risk	Matches finding
SQL Injection — Authentication Bypass	High	V-01
Cross Site Scripting on /profile	High	V-02
Cookie No HttpOnly Flag	Low	V-06
Cookie Without SameSite Attribute	Low	V-06
CSP Header Not Set	Medium	V-04

ZAP alert	Risk	Matches finding
Missing Anti-clickjacking Header	Medium	V-04
X-Content-Type-Options Header Missing	Low	V-04
Server Leaks Version via "X-Powered-By"	Low	V-05
Absence of Anti-CSRF Tokens	Medium	V-06

2.3 Findings against secure-app/

After applying the Week 2 fixes, the same Automated Scan against **secure-app/** produces only informational alerts (Modern Web Application, Storable but Non-Cacheable Content). All High and Medium alerts from the vulnerable scan are gone.

3 Manual Attacks — curl and Browser Dev Tools

3.1 SQL injection authentication bypass

```
curl -i -X POST http://localhost:3000/login \
  --data-urlencode "username=admin' OR '1'='1" \
  --data-urlencode "password=anything"
```

Vulnerable build: 302 Found, Set-Cookie: uid=1, Location: /profile — bypassed.

Secure build: 200 OK with "Invalid credentials".

3.2 Stored XSS in bio

1. Login to the vulnerable app.
2. Open Dev Tools → **Elements** to confirm the bio container.
3. Set bio to `<script>alert('XSS');</script>` and save.
4. Reload /profile.

Vulnerable build: alert fires. The cookie is readable from the console with `document.cookie` because the `HttpOnly` flag is missing.

Secure build: the payload renders as escaped text, and the CSP would block it even if escaping had been forgotten.

3.3 Cookie tampering and privilege escalation

In the vulnerable app, run in the dev-tools console:

```
document.cookie = "uid=1; path=/";
location.reload();
```

Vulnerable build: logged in as user id 1 (admin). The cookie has no signature.

Secure build: the cookie is a signed JWT. Tampering breaks the signature, the server treats the request as anonymous, and `security.log` records "Rejected invalid/expired token".

3.4 Brute force and credential stuffing

```
for i in $(seq 1 25); do
  curl -s -o /dev/null -w "%{http_code}\n" \
    -X POST http://localhost:3000/login \
    --data-urlencode "username=admin" \
```

```
--data-urlencode "password=wrong$i"
done
```

Vulnerable build: 25 attempts return 200 (“Invalid credentials”) — no limit.

Secure build: 20 attempts return 200, then 429 Too Many Requests for the remaining attempts.

4 Browser Dev-Tools Checklist

Use this checklist when assessing any other web app:

- **Network tab.** Inspect Set-Cookie for HttpOnly, Secure, and SameSite.
- **Response headers.** Look for Content-Security-Policy, Strict-Transport-Security, X-Frame-Options, and Referrer-Policy.
- **Console.** Run `document.cookie` to see what JS can reach. Try editing cookies and `localStorage`.
- **Application tab.** Check `localStorage` for sensitive data.
- **Elements tab.** Confirm where user input is reflected in the DOM and whether it is HTML-escaped.

5 Before and After Summary

Attack	vulnerable-app	secure-app
SQLi auth bypass	passes (logged in as admin)	blocked (“Invalid credentials”)
Stored XSS in bio	passes (alert fires)	blocked (rendered as text + CSP)
Cookie tampering	passes (set <code>uid=1</code>)	blocked (JWT signature fails)
Brute force	passes (unlimited)	blocked (429 after 20 / 15 min)
Header scan	passes (no headers)	blocked (full helmet set)
<code>/users</code> enumeration	passes (leaks emails)	blocked (requires login)